ADDENDUM — DATABASE-PER-SCHOOL ON NEON (POSTGRES)

Supersedes Sections 2 & 4 of the FINAL PLAN document

> **Trigger for this addendum:** switching from a shared MongoDB (tenantId-scoped) architecture to **one Neon Postgres project per school**, each with its own API integrations (M-Pesa paybill, SMS credentials). Starting with one school; designed so school #2+ is a repeatable, not a redesign.

A. WHY THIS CHANGES THE ARCHITECTURE (Read Before Coding)

The original plan used a **shared database, row-level isolation** model (`tenantId` on every document, enforced by middleware). That model exists specifically to solve the problem of many tenants sharing one database safely.

You're now choosing **physical isolation instead**: one Neon project = one school's entire database. This is actually a stronger isolation guarantee (there is no query that *can* cross tenants — there's no shared table to query across), but it trades the row-level-scoping problem for a different one: **you now need a place that knows which school maps to which Neon project and which API credentials**, and a plan for what happens when you have more than one school.

Resolved decision for now (one school): don't build that control-plane layer yet. Build a single-tenant app, fully configured via environment variables, pointed at one Neon project. Structure config so that "add school #2" is: (1) create a new Neon project, (2) copy the env template, (3) deploy a second instance (or same codebase on a new subdomain). No school registry, no cross-school routing logic, no premature abstraction.

When you actually add school #2 or #3, the only new piece of infrastructure is a tiny control-plane table (can live in its own small Neon project) with columns like `{ schoolCode, neonProjectConnectionString(secret ref), mpesaCredsRef, smsCredsRef, subdomain, isActive }`, consulted once per incoming request to resolve which DB/credentials to use for that request. That is a Phase-2-of-growth concern, not a Phase-1 concern — don't build it now, but don't design anything in Phase 1 that would make it painful to add later (e.g. don't hardcode a single global env var name for the DB URL inside business logic — read it from one central config module so swapping to per-request resolution later touches one file).

B. NEON-SPECIFIC CONSTRAINTS TO DESIGN AROUND

Verified current (2026) free-tier numbers — re-check before you scale past school #1, as usage-based platforms revise limits:

Constraint	Free tier value	Design implication
Storage	0.5 GB / project	Plenty for one school's operational data (thousands of students/invoices/logs) for years; keep large binary blobs (photos, receipts PDFs) in object storage (Cloudinary/S3), never in Postgres.
Compute	100 CU-hours/month, autoscale to 2 CU	Fine for a school admin app's traffic pattern. Cron jobs (SMS automated rules, nightly reconciliation) consume this budget too — schedule them, don't poll constantly.
Scale-to-zero	Mandatory, ~300–500ms cold start	Cron/webhook endpoints (M-Pesa callback, SMS delivery webhook) will sometimes hit a cold DB. Design webhook handlers to tolerate the extra latency; Safaricom's webhook timeout budget still needs to be respected — see idempotency note below.
Connections	Pooled endpoint supports up to 10,000 pooled connections	**Always use Neon's pooled connection string** (not the direct one) from the Node app, or use the `@neondatabase/serverless` driver over HTTP. Traditional `pg`/Prisma connection pooling on top of an already-pooled Neon endpoint can double-pool and cause issues — use Neon's pooler *instead of* your own pool, not in addition to it.
Branches	Up to 10 per project on free tier	Use one branch per environment: `main` (production), `dev`, `test`. Free — use them for schema migration testing before hitting production.

Practical gotcha: if you use polling (e.g. a frontend health-check or a naive "keep the connection warm" trick), you defeat scale-to-zero and burn your compute-hour budget for nothing. Don't do this — let it sleep.

C. ORM CHOICE

Two reasonable options for Node/Express + Neon; pick one and commit — don't mix:

- **Drizzle ORM** — lighter weight, SQL-like syntax, first-class support for the Neon HTTP/serverless driver, faster cold starts (matters given scale-to-zero). **Recommended** for this project given the serverless-leaning deployment.
- **Prisma** — more tooling/ergonomics (studio, migration UX), works fine with Neon's pooled connection string, slightly heavier cold-start footprint.

Rest of this addendum assumes Drizzle-style schema definitions, but the table design translates directly to Prisma if you prefer it.

D. DATA MODELS — CONVERTED TO POSTGRES (Relational, Per-School DB)

Since each school has its own database, **no `tenantId` column is needed anywhere below** — the database boundary *is* the tenant boundary. This is a meaningful simplification vs. the Mongo version.

sql

```sql
-- ============ CORE / AUTH ============

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  email TEXT UNIQUE NOT NULL,
  phone TEXT,
  password_hash TEXT NOT NULL,
  role TEXT NOT NULL CHECK (role IN ('schoolAdmin','teacher','accountant','pare
  permissions TEXT[] DEFAULT '{}',
  is_active BOOLEAN DEFAULT true,
  must_reset_password BOOLEAN DEFAULT false,
  last_login_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);
-- Note: no Super Admin role/table here — Super Admin lives OUTSIDE this per-sc
-- entirely (it's a cross-school concern; see Section A control-plane note).

-- ============ STUDENTS & GUARDIANS ============

CREATE TABLE students (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  admission_number TEXT UNIQUE NOT NULL,
  first_name TEXT NOT NULL,
  last_name TEXT NOT NULL,
  date_of_birth DATE,
  gender TEXT,
  grade TEXT NOT NULL,
  class TEXT NOT NULL,
  enrollment_date DATE NOT NULL,
  status TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active','transferred
  photo_url TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- Guardians as a proper table, not an embedded array — this is the natural
-- Postgres equivalent of the guardians[] fix from the Mongo version, and it's
-- actually cleaner here: relational FK instead of subdocument.
CREATE TABLE guardians (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  student_id UUID NOT NULL REFERENCES students(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  phone TEXT NOT NULL,
  relationship TEXT,
```

```sql
  is_primary BOOLEAN DEFAULT false,
  preferred_language TEXT DEFAULT 'en' CHECK (preferred_language IN ('sw','en')
  sms_opt_out BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_guardians_student ON guardians(student_id);
CREATE INDEX idx_guardians_phone ON guardians(phone); -- lookups for two-way SM

-- ============ ACADEMIC (CBC) ============

CREATE TABLE learning_areas (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  grade_levels TEXT[] NOT NULL
);

CREATE TABLE strands (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  learning_area_id UUID NOT NULL REFERENCES learning_areas(id) ON DELETE CASCAD
  name TEXT NOT NULL
);

CREATE TABLE assessments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  student_id UUID NOT NULL REFERENCES students(id) ON DELETE CASCADE,
  learning_area_id UUID NOT NULL REFERENCES learning_areas(id),
  strand_id UUID REFERENCES strands(id),
  term SMALLINT NOT NULL CHECK (term IN (1,2,3)),
  academic_year SMALLINT NOT NULL,
  type TEXT NOT NULL CHECK (type IN ('formative','summative')),
  performance_level SMALLINT CHECK (performance_level BETWEEN 1 AND 4),
  score NUMERIC(5,2),
  remarks TEXT,
  assessed_by UUID REFERENCES users(id),
  assessed_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_assessments_student_term ON assessments(student_id, term, acad

CREATE TABLE report_cards (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  student_id UUID NOT NULL REFERENCES students(id) ON DELETE CASCADE,
  term SMALLINT NOT NULL,
  academic_year SMALLINT NOT NULL,
  compiled_results JSONB NOT NULL, -- per-learning-area rollup; JSONB is fine h
  teacher_comments TEXT,
  head_teacher_comments TEXT,
  generated_at TIMESTAMPTZ DEFAULT now(),
```

```
  generated_by UUID REFERENCES users(id),
  published_at TIMESTAMPTZ -- NULL until officially released; SMS/portal trigge
);
CREATE UNIQUE INDEX idx_report_card_unique ON report_cards(student_id, term, ac

-- ============ FEES & M-PESA ============

CREATE TABLE fee_structures (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  grade TEXT NOT NULL,
  term SMALLINT NOT NULL,
  academic_year SMALLINT NOT NULL,
  items JSONB NOT NULL, -- [{name, amount}]
  total_amount NUMERIC(10,2) NOT NULL
);

CREATE TABLE fee_invoices (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  student_id UUID NOT NULL REFERENCES students(id) ON DELETE CASCADE,
  fee_structure_id UUID NOT NULL REFERENCES fee_structures(id),
  term SMALLINT NOT NULL,
  academic_year SMALLINT NOT NULL,
  total_amount NUMERIC(10,2) NOT NULL,
  amount_paid NUMERIC(10,2) NOT NULL DEFAULT 0,
  balance NUMERIC(10,2) GENERATED ALWAYS AS (total_amount - amount_paid) STORED
  due_date DATE,
  status TEXT NOT NULL DEFAULT 'unpaid' CHECK (status IN ('unpaid','partial','p
  last_reminder_date DATE
);
CREATE INDEX idx_invoice_balance ON fee_invoices(balance) WHERE balance > 0; --

CREATE TABLE paybills (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  shortcode TEXT NOT NULL UNIQUE,
  account_reference_format TEXT NOT NULL, -- e.g. '{admissionNumber}'
  purpose TEXT NOT NULL CHECK (purpose IN ('tuition','trip','event','custom')),
  daraja_consumer_key_ref TEXT NOT NULL, -- reference into a secrets manager, N
  daraja_consumer_secret_ref TEXT NOT NULL,
  daraja_passkey_ref TEXT NOT NULL,
  is_active BOOLEAN DEFAULT true
);

CREATE TABLE mpesa_transactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  paybill_id UUID NOT NULL REFERENCES paybills(id),
  transaction_id TEXT NOT NULL UNIQUE, -- Safaricom receipt number = idempotenc
```

```
  amount NUMERIC(10,2) NOT NULL,
  phone_number TEXT NOT NULL,
  account_reference TEXT NOT NULL,
  transaction_date TIMESTAMPTZ NOT NULL,
  matched_student_id UUID REFERENCES students(id),
  matched_invoice_id UUID REFERENCES fee_invoices(id),
  match_status TEXT NOT NULL DEFAULT 'unmatched' CHECK (match_status IN ('auto-
  raw_payload JSONB NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);
-- The UNIQUE constraint on transaction_id is your webhook-retry guard.
-- Insert with `ON CONFLICT (transaction_id) DO NOTHING` and check rowcount ==
-- to detect (and safely ignore) a duplicate Safaricom callback delivery.

CREATE TABLE receipts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  invoice_id UUID NOT NULL REFERENCES fee_invoices(id),
  mpesa_transaction_id UUID REFERENCES mpesa_transactions(id),
  receipt_number SERIAL UNIQUE, -- Postgres auto-increment gives you sequential
  amount NUMERIC(10,2) NOT NULL,
  payment_method TEXT NOT NULL CHECK (payment_method IN ('mpesa','cash','bank')
  issued_at TIMESTAMPTZ DEFAULT now(),
  issued_by UUID REFERENCES users(id)
);

-- ============ STAFF & PAYROLL ============

CREATE TABLE staff (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  phone TEXT,
  email TEXT,
  role TEXT NOT NULL CHECK (role IN ('teacher','support')),
  employment_type TEXT NOT NULL CHECK (employment_type IN ('flatRate','sessionB
  base_rate NUMERIC(10,2) NOT NULL,
  payout_ref TEXT, -- reference to encrypted bank/mpesa payout details, stored
  is_active BOOLEAN DEFAULT true
);

CREATE TABLE attendance_staff (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  staff_id UUID NOT NULL REFERENCES staff(id) ON DELETE CASCADE,
  date DATE NOT NULL,
  sessions_taught SMALLINT DEFAULT 0,
  status TEXT NOT NULL CHECK (status IN ('present','absent','halfDay','leave'))
  UNIQUE(staff_id, date) -- prevents double-marking the same person twice in a
);
```

```sql
CREATE TABLE payroll_runs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  month SMALLINT NOT NULL,
  year SMALLINT NOT NULL,
  status TEXT NOT NULL DEFAULT 'draft' CHECK (status IN ('draft','approved','pa
  approved_by UUID REFERENCES users(id),
  approved_at TIMESTAMPTZ,
  paid_at TIMESTAMPTZ,
  UNIQUE(month, year)
);

CREATE TABLE payroll_entries (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  payroll_run_id UUID NOT NULL REFERENCES payroll_runs(id) ON DELETE CASCADE,
  staff_id UUID NOT NULL REFERENCES staff(id),
  base_rate NUMERIC(10,2) NOT NULL,
  sessions_counted SMALLINT,
  days_present SMALLINT,
  gross_pay NUMERIC(10,2) NOT NULL,
  net_pay NUMERIC(10,2) NOT NULL
);

CREATE TABLE payroll_adjustments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  payroll_entry_id UUID NOT NULL REFERENCES payroll_entries(id) ON DELETE CASCA
  reason TEXT NOT NULL,
  amount NUMERIC(10,2) NOT NULL -- positive = bonus, negative = deduction; alwa
);
-- Application-layer rule (enforce in the service, not just the UI):
-- payroll_entries rows for a run may only be inserted/updated while
-- payroll_runs.status = 'draft'. Once 'approved', treat as immutable;
-- corrections go in as a NEW payroll_adjustments row on a later run, never
-- an UPDATE to an approved entry.

-- ============ LEDGER ============

CREATE TABLE ledger_accounts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  code TEXT UNIQUE NOT NULL,
  name TEXT NOT NULL,
  type TEXT NOT NULL CHECK (type IN ('asset','liability','equity','income','exp
);

CREATE TABLE ledger_entries (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  entry_date DATE NOT NULL,
```

```
  description TEXT,
  source_type TEXT NOT NULL CHECK (source_type IN ('feePayment','payroll','expe
  source_id UUID,
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE ledger_lines (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ledger_entry_id UUID NOT NULL REFERENCES ledger_entries(id) ON DELETE CASCADE
  account_id UUID NOT NULL REFERENCES ledger_accounts(id),
  debit NUMERIC(10,2) NOT NULL DEFAULT 0,
  credit NUMERIC(10,2) NOT NULL DEFAULT 0,
  CHECK (debit >= 0 AND credit >= 0),
  CHECK (NOT (debit > 0 AND credit > 0)) -- a line is either a debit OR a credi
);

-- Postgres CHECK constraints can't aggregate across sibling rows, so the
-- "sum(debit) == sum(credit) per entry" rule needs a trigger, not a CHECK:
CREATE OR REPLACE FUNCTION enforce_balanced_ledger_entry() RETURNS TRIGGER AS $
DECLARE
  total_debit NUMERIC(10,2);
  total_credit NUMERIC(10,2);
BEGIN
  SELECT COALESCE(SUM(debit),0), COALESCE(SUM(credit),0)
    INTO total_debit, total_credit
    FROM ledger_lines WHERE ledger_entry_id = NEW.ledger_entry_id;
  IF total_debit != total_credit THEN
    RAISE EXCEPTION 'Ledger entry % is unbalanced: debit % != credit %',
      NEW.ledger_entry_id, total_debit, total_credit;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER trg_balanced_ledger
  AFTER INSERT OR UPDATE ON ledger_lines
  DEFERRABLE INITIALLY DEFERRED -- lets you insert all lines of one entry in a
  FOR EACH ROW EXECUTE FUNCTION enforce_balanced_ledger_entry();

-- ============ ATTENDANCE (STUDENT) ============

CREATE TABLE attendance_student (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  student_id UUID NOT NULL REFERENCES students(id) ON DELETE CASCADE,
  date DATE NOT NULL,
  status TEXT NOT NULL CHECK (status IN ('present','absent','late','excused')),
```

```
  marked_by UUID REFERENCES users(id),
  UNIQUE(student_id, date)
);

-- ============ TIMETABLE ============

CREATE TABLE timetable_slots (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  grade TEXT NOT NULL,
  class TEXT NOT NULL,
  day_of_week SMALLINT NOT NULL CHECK (day_of_week BETWEEN 1 AND 7),
  start_time TIME NOT NULL,
  end_time TIME NOT NULL,
  learning_area_id UUID REFERENCES learning_areas(id),
  staff_id UUID REFERENCES staff(id),
  room TEXT,
  CHECK (end_time > start_time)
);
-- Overlap prevention needs Postgres's EXCLUDE constraint (requires btree_gist
CREATE EXTENSION IF NOT EXISTS btree_gist;
ALTER TABLE timetable_slots ADD CONSTRAINT no_staff_overlap
  EXCLUDE USING gist (
    staff_id WITH =,
    day_of_week WITH =,
    tsrange(start_time::text::timestamp, end_time::text::timestamp) WITH &&
  ) WHERE (staff_id IS NOT NULL);
ALTER TABLE timetable_slots ADD CONSTRAINT no_class_overlap
  EXCLUDE USING gist (
    class WITH =,
    day_of_week WITH =,
    tsrange(start_time::text::timestamp, end_time::text::timestamp) WITH &&
  );
-- This makes timetable conflicts IMPOSSIBLE at the database level, not just
-- caught by application code — a stronger guarantee than the Mongo version had

-- ============ SMS ENGINE (adapter-agnostic, per Section 5 of main plan) =====

CREATE TABLE sms_templates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  type TEXT NOT NULL CHECK (type IN ('feeReminder','receipt','results','meeting
  content TEXT NOT NULL,
  variables TEXT[] DEFAULT '{}',
  language TEXT DEFAULT 'en' CHECK (language IN ('sw','en')),
  is_default BOOLEAN DEFAULT false,
  is_active BOOLEAN DEFAULT true
);
```

```
CREATE TABLE sms_campaigns (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  type TEXT NOT NULL CHECK (type IN ('bulk','scheduled','automated')),
  target_filter JSONB NOT NULL,
  template_id UUID REFERENCES sms_templates(id),
  message TEXT,
  recipient_count INT,
  status TEXT NOT NULL DEFAULT 'draft' CHECK (status IN ('draft','scheduled','s
  scheduled_at TIMESTAMPTZ,
  sent_at TIMESTAMPTZ,
  sent_by UUID REFERENCES users(id),
  cost NUMERIC(10,2)
);

CREATE TABLE sms_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  campaign_id UUID REFERENCES sms_campaigns(id),
  recipient_id UUID, -- guardian_id, teacher's user_id, or staff_id depending o
  recipient_type TEXT NOT NULL CHECK (recipient_type IN ('guardian','teacher','
  phone_number TEXT NOT NULL,
  message TEXT NOT NULL,
  template_id UUID REFERENCES sms_templates(id),
  provider_message_id TEXT, -- opaque ID from YOUR custom SMS solution
  status TEXT NOT NULL DEFAULT 'queued' CHECK (status IN ('queued','sent','deli
  error_message TEXT,
  sent_at TIMESTAMPTZ,
  delivered_at TIMESTAMPTZ,
  cost NUMERIC(10,2)
);

CREATE TABLE automated_sms_rules (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  trigger_type TEXT NOT NULL CHECK (trigger_type IN ('feeDueSoon','feeOverdue',
  conditions JSONB NOT NULL,
  template_id UUID REFERENCES sms_templates(id),
  is_active BOOLEAN DEFAULT true,
  last_run_at TIMESTAMPTZ
);
```

What got structurally better by moving to Postgres, not just translated:

- Timetable conflicts are now prevented by an `EXCLUDE` constraint — the database physically rejects an overlapping slot. In the Mongo version this was "must be

enforced server-side" application logic (i.e., trustable only if every code path remembers to call it). Now it can't be bypassed.

- Ledger balance is enforced by a deferred constraint trigger, not just a pre-save hook — same effect, but it fires even if some future code path writes `ledger_lines` directly instead of going through your service (defense in depth).
- `fee_invoices.balance` is a Postgres `GENERATED ALWAYS AS` column — it is *computed by the database itself*, so "trusting a stale cached balance" (the bug class flagged in the original plan) becomes structurally impossible rather than something you have to remember to recompute.
- `mpesa_transactions.transaction_id UNIQUE` + `ON CONFLICT DO NOTHING` gives you the webhook-retry idempotency guard as a one-line insert pattern instead of app-level check-then-write logic (which has its own race condition if two webhook deliveries arrive concurrently — the DB constraint closes that race).

E. WHERE SUPER ADMIN / MULTI-SCHOOL FITS (Not Now — Just So You Don't Paint Yourself In)

You don't need this for school #1. When you add school #2:

1. Create a minimal separate Neon project (or a lightweight Postgres elsewhere) holding just a `schools` control table: `{ id, name, subdomain, db_connection_ref, mpesa_creds_ref, sms_creds_ref, subscription_status }`.
2. Your app's request-handling layer resolves `subdomain → school row → DB connection` once per request (cache this lookup, it rarely changes) and opens a connection to that school's Neon project for the rest of the request.
3. Super Admin dashboard queries the control table for the roster/subscription view, and — if you ever need cross-school analytics — either queries each school's DB one at a time and aggregates in application code, or has each school's app periodically push a small rollup (e.g. "42 students, KES 12,000 collected this term") into the control-plane DB rather than trying to run cross-database SQL (Postgres doesn't support cross-database joins natively, even within the same Neon account).

This is a small, isolated addition later — nothing in Section D's schema needs to change to support it.

F. REVISED PHASE 1 (Concrete, Neon-Specific)

Replaces the generic "Phase 1: Foundation" row from the main plan:

1. Create one Neon project for the school. Note the pooled connection string (not the direct one).

2. Set up Drizzle (or Prisma) with the schema in Section D, run the initial migration against a `dev` branch first, then `main`.
3. Wire the app's config so the DB connection string, Daraja credentials, and your SMS solution's credentials are all read from environment variables via one central config module — not scattered `process.env` reads across the codebase (this is the one thing that makes adding school #2 painless later).
4. Auth + RBAC against the `users` table.
5. Confirm scale-to-zero behavior is acceptable: hit the app cold after 10+ minutes idle, measure the actual latency you get, decide if that's fine for your admins/parents or if you want a cheap external cron pinging it during school hours only (weigh this against burning free compute-hours for no user benefit).

This addendum is authoritative over Sections 2 and 4 of the original FINAL PLAN document wherever they conflict. Sections 1, 3, 5, 6, 7, 9 (adjusted per F above), and 10 of the original plan still apply — they described business logic and module scope, which doesn't change with the database engine.